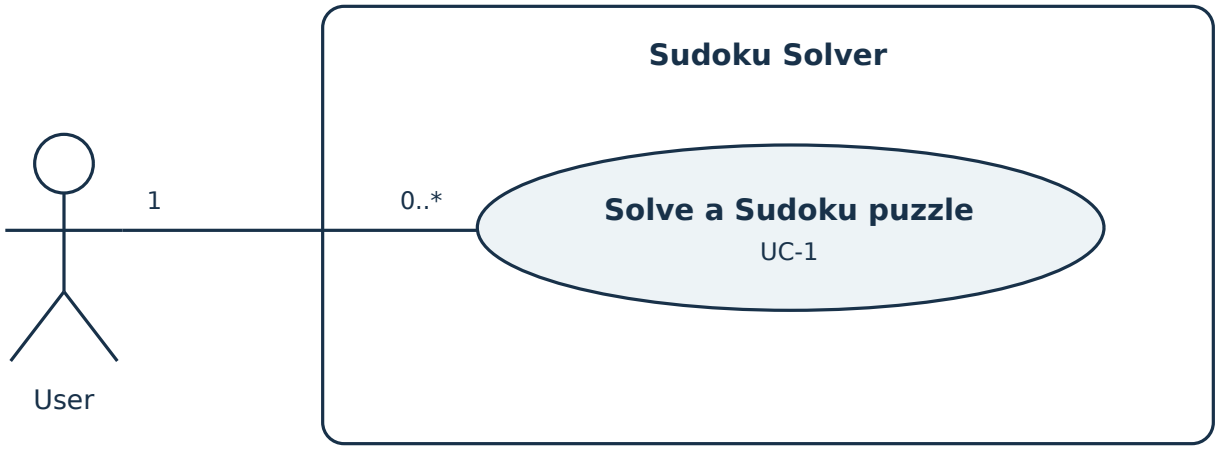
Sudoku use-case model

1. Identify the actor, system, and use case

Actor: User, such as a student or grader running the program.
System: Sudoku Solver.
Use case UC-1: Solve a Sudoku puzzle.
User goal: Obtain every valid solution to a supplied puzzle, or learn that no solution exists.



Each execution involves one user. A user may run the program zero or more times. Reading files, checking cells, and printing results are steps within this user goal. Puzzle text files are input data, not separate actors.

2. Brief description

The user supplies one puzzle filename when starting the solver, for example ./Assignment4.exe puzzle1.txt. The system displays the filename and original puzzle, then prints every solution or "No solution found" and the total count. The user runs the program separately for other puzzles.

3. Step-by-step flow

1. The user makes a puzzle file available and opens a terminal in the executable directory.

2. The user starts the solver with one filename: ./Assignment4.exe puzzle1.txt.

3. The system checks that exactly one filename was supplied, then opens it and reads 81 cells.

4. The system displays the filename and original puzzle.

5. The system checks the clues and searches for every valid completion.

6. The system displays each solution and the total solution count.

7. The system terminates; the user reviews the results and may run it again for another file.

4. Detailed description

Field	Description
Actor and trigger	The user starts the compiled Sudoku Solver with one puzzle filename.
Preconditions	For normal processing, the named file is readable and contains exactly 81 cells: digits 1-9 or underscores, with whitespace ignored.
Postconditions	All solutions or a no-solution message are displayed. Argument or file errors are reported. The input file is unchanged.

Detailed normal flow

1. Start the solver
The user enters ./Assignment4.exe followed by one filename. The system checks the argument count. If it is incorrect, the system displays usage instructions and exits.

2. Load and display the puzzle
The system opens the named file and reads exactly 81 cells, accepting digits 1-9 and underscores while ignoring whitespace. It displays the filename and original grid. An unreadable or incorrectly formatted file produces an error and ends the program.

3. Check the starting clues
The system checks each given digit against its row, column, and block. Conflicting clues lead to the no-solution result without searching.

4. Find and display solutions
For consistent clues, the system explores all allowed completions while preserving the given digits. It prints each completed solution with a number. When searching ends, it displays the total or reports that no solution was found.

5. End the execution
The program exits. The user reviews the results and can start a separate execution with another filename. The input file remains unchanged.

Alternate and exception flows

Branch	Observable behavior
A1 - No solution	If clues conflict or no valid completion exists, print "No solution found" and "Solutions found: 0," then terminate with status 0.
A2 - Multiple solutions	Print every distinct solution, numbered in discovery order. Do not stop after the first. Print the final count.
E1 - Missing/unreadable file	Print the filename and a file/input error. Terminate with exit status 1.
E2 - Invalid file contents	If a file has an invalid character or a cell count other than 81, report an input error and terminate with exit status 1.
E3 - Wrong argument count	If zero or multiple filenames are supplied, print usage instructions and terminate with exit status 1.
A3 - Already completed puzzle	If a fully filled grid is valid, print it as one solution. If it violates Sudoku rules, use A1.